

# GUÍA DE REPASO COMPLETA

Módulo 1 · Unidad 1 · Clase 3

## SQL Aplicado: Extracción y Análisis de Datos Relacionales

### ¿Para quién es esta guía?

Esta guía cubre SQL desde cero: SELECT, WHERE, GROUP BY, HAVING y JOINs. Está diseñada para que puedas escribir consultas sobre bases de datos reales sin conocimiento previo de SQL. Cada cláusula se explica conceptualmente antes del código, y cada resultado se interpreta en términos técnicos y de negocio.

**SQL (Structured Query Language)** es el lenguaje estándar para interactuar con bases de datos relacionales. Aunque Python y Pandas dominan el análisis exploratorio, en la práctica la mayoría de los datos de una empresa viven en bases de datos SQL. Saber extraerlos correctamente es una habilidad no negociable en ciencia de datos.

## SECCIÓN 0 | Mapa Conceptual de la Clase

Estos son los cinco bloques de SQL de esta clase y cómo se encadenan:

| Bloque                             | Qué hace y cuándo se usa                                                                                                                                  |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| SELECT / FROM                      | El punto de partida de toda consulta. Define qué columnas quieres ver y de qué tabla. Sin SELECT y FROM no hay consulta posible.                          |
| WHERE                              | Filtra filas según una condición. Si SELECT/FROM es 'dame la tabla', WHERE es 'pero solo las filas que cumplan esto'.                                     |
| GROUP BY + funciones de agregación | Agrupar filas con el mismo valor en una columna y calcular estadísticas por grupo (SUM, COUNT, AVG, MIN, MAX). Transforma datos fila a fila en resúmenes. |
| HAVING                             | Filtra grupos, no filas individuales. Es el WHERE que se aplica después del GROUP BY. No puede usarse sin GROUP BY.                                       |
| JOINs (INNER, LEFT)                | Combina información de dos o más tablas usando una clave en común. Sin JOINs, los datos relacionales son solo fragmentos aislados.                        |

### La lógica de una consulta SQL completa

El orden de ejecución lógica de SQL (que es distinto al orden en que se escribe) es: FROM → WHERE → GROUP BY → HAVING → SELECT. La base de datos primero elige la tabla (FROM), filtra filas (WHERE), agrupa (GROUP BY), filtra grupos (HAVING), y finalmente selecciona las columnas (SELECT). Entender este orden evita muchos errores de lógica.

## SECCIÓN 1 | Bases Relacionales: Por Qué SQL

### 1.1 ¿Qué es una base de datos relacional?

Una base de datos relacional organiza la información en tablas (también llamadas relaciones), donde cada tabla tiene filas (registros) y columnas (atributos). La clave del modelo relacional es que las tablas se conectan entre sí mediante claves: una clave primaria (Primary Key) identifica de manera única cada fila de una tabla, y una clave foránea (Foreign Key) apunta a la clave primaria de otra tabla.

Esta estructura evita la duplicación de datos. En lugar de guardar el nombre del origen y destino de un vuelo en cada registro de reserva, se guarda un `id_ruta` que apunta a la tabla de rutas. Si la ruta cambia su nombre, se modifica en un solo lugar.

### 1.2 SQLite y Python: sin instalación adicional

Para esta guía usamos [SQLite](#), el motor de base de datos incluido en Python mediante el módulo [sqlite3](#). No requiere instalar ningún servidor ni paquete adicional. En proyectos reales encontrarás PostgreSQL, MySQL o SQL Server, pero las consultas que aprenderás aquí funcionan en todos esos motores con mínimas diferencias de sintaxis.

```
import sqlite3

# Conectar a una base de datos en memoria (temporal, no genera archivos)
conn = sqlite3.connect(':memory:')
cur = conn.cursor()

print('Conexión establecida con SQLite')
```

---

## SECCIÓN 2 | Caso Desarrollado: Base de Datos Operativa de Avianca

### Contexto verificado: Avianca

Avianca es la aerolínea más antigua del hemisferio occidental y la segunda más antigua del mundo en operación continua, después de KLM. Fue fundada el 5 de diciembre de 1919 en Barranquilla bajo el nombre de Sociedad Colombo-Alemana de Transportes Aéreos (SCADTA), por un grupo de empresarios colombianos y alemanes que incluyó a Ernesto Cortissoz, Werner Kaemmerer y Alberto Tietjen. En 1940 adoptó el nombre Avianca (Aerovías Nacionales de Colombia). En 2023 transportó más de 32.2 millones de pasajeros

con más de 213.000 vuelos. Su programa de lealtad LifeMiles supera los 14 millones de miembros.

Caso de uso: el equipo de analítica de Avianca mantiene una base de datos relacional con información de rutas, vuelos, pasajeros y reservas. Tú eres el analista de datos recién contratado. Tu primera tarea: extraer información clave usando SQL para responder preguntas de negocio concretas.

## Paso 0: Crear el esquema y los datos

Antes de consultar, definimos las cuatro tablas de nuestra base de datos y las poblamos con datos de prueba. Cada tabla tiene una clave primaria (id\_\*) y las relaciones entre tablas se establecen con claves foráneas.

```
cur.executescript('''
    CREATE TABLE rutas (
        id_ruta      INTEGER PRIMARY KEY,
        origen       TEXT NOT NULL,
        destino      TEXT NOT NULL,
        distancia_km INTEGER NOT NULL
    );

    CREATE TABLE vuelos (
        id_vuelo      INTEGER PRIMARY KEY,
        id_ruta       INTEGER NOT NULL,
        fecha         TEXT NOT NULL,
        precio_cop    INTEGER NOT NULL,
        asientos_vendidos INTEGER NOT NULL,
        asientos_totales INTEGER NOT NULL,
        FOREIGN KEY (id_ruta) REFERENCES rutas(id_ruta)
    );

    CREATE TABLE pasajeros (
        id_pasajero  INTEGER PRIMARY KEY,
        nombre       TEXT NOT NULL,
        categoria    TEXT NOT NULL,
        millas       INTEGER NOT NULL
    );

    CREATE TABLE reservas (
        id_reserva   INTEGER PRIMARY KEY,
        id_vuelo     INTEGER NOT NULL,
        id_pasajero  INTEGER NOT NULL,
        clase        TEXT NOT NULL,
        FOREIGN KEY (id_vuelo) REFERENCES vuelos(id_vuelo),
```

```

        FOREIGN KEY (id_pasajero) REFERENCES pasajeros(id_pasajero)
    );
'''

```

### Interpretación: Esquema

**Interpretación técnica:** NOT NULL garantiza que ningún campo obligatorio quede vacío. FOREIGN KEY crea la relación entre tablas: un vuelo sin ruta válida no puede existir en la base de datos. Este tipo de restricciones son la razón por la que los datos en producción son más confiables que los datos en hojas de cálculo.

**Interpretación de negocio:** este esquema refleja la realidad de Avianca: una ruta puede tener muchos vuelos (relación 1:N entre rutas y vuelos), y un vuelo puede tener muchos pasajeros (relación N:M entre vuelos y pasajeros, materializada en la tabla reservas).

## Paso 1: SELECT y FROM — la consulta más básica

SELECT define qué columnas quieres ver. FROM define de qué tabla las tomas. El asterisco (\*) selecciona todas las columnas. En producción, es mejor nombrar las columnas explícitamente para claridad y rendimiento.

```

# Consulta 1: ver todas las rutas
cur.execute('SELECT id_ruta, origen, destino, distancia_km FROM rutas')
resultados = cur.fetchall()
for fila in resultados:
    print(fila)

```

```

(1, 'Bogotá', 'Medellín', 232)
(2, 'Bogotá', 'Cali', 433)
(3, 'Bogotá', 'Cartagena', 1029)
(4, 'Medellín', 'Cartagena', 829)
(5, 'Bogotá', 'Barranquilla', 1062)

```

### Interpretación: Paso 1

**Interpretación técnica:** fetchall() recupera todos los resultados de la consulta como una lista de tuplas. Cada tupla es una fila. El orden de las columnas en la tupla corresponde al orden del SELECT.

**Interpretación de negocio:** Avianca opera 5 rutas domésticas en este dataset. La ruta más larga es Bogotá-Barranquilla (1.062 km). Nótese que la ruta Medellín-Cartagena (829 km) está registrada pero Barranquilla-Bogotá no: las rutas de vuelta se tratan como rutas separadas en muchos sistemas.

## Paso 2: WHERE — filtrar filas con condiciones

WHERE reduce el conjunto de resultados a solo las filas que cumplen una condición. Se puede combinar con AND, OR y NOT. Las condiciones pueden usar operadores de comparación (=, >, <, >=, <=, !=) y operadores de texto como LIKE.

```
# Consulta 2: vuelos con precio superior a 400.000 COP
cur.execute('''
    SELECT id_vuelo, id_ruta, fecha, precio_cop
    FROM    vuelos
    WHERE   precio_cop > 400000
    ORDER BY precio_cop DESC
''')
for fila in cur.fetchall():
    print(fila)
```

```
(8, 5, '2024-03-01', 510000)
(9, 5, '2024-03-05', 495000)
(6, 3, '2024-03-04', 480000)
(5, 3, '2024-03-01', 450000)
```

```
# Consulta 3: vuelos de la ruta 1 (Bogotá-Medellín) con más de 140
asientos vendidos
cur.execute('''
    SELECT id_vuelo, fecha, asientos_vendidos
    FROM    vuelos
    WHERE   id_ruta = 1 AND asientos_vendidos > 140
''')
for fila in cur.fetchall():
    print(fila)
```

```
(1, '2024-03-01', 142)
(2, '2024-03-02', 156)
```

### Interpretación: Paso 2

**Interpretación técnica:** ORDER BY DESC ordena los resultados de mayor a menor. La condición AND requiere que se cumplan ambas condiciones simultáneamente. Con OR bastaría una de las dos.

**Interpretación de negocio:** los 4 vuelos más costosos corresponden a las rutas 3 (Bogotá-Cartagena) y 5 (Bogotá-Barranquilla), que son las de mayor distancia. En la ruta Bogotá-Medellín, solo 2 de 3 vuelos superaron los 140 pasajeros, lo que puede indicar variación en

la demanda según el día.

### Paso 3: GROUP BY y funciones de agregación

GROUP BY agrupa las filas que tienen el mismo valor en una columna y luego aplica funciones de agregación a cada grupo: COUNT (contar), SUM (sumar), AVG (promedio), MIN (mínimo), MAX (máximo). Es la cláusula que transforma datos individuales en resúmenes de negocio.

Regla fundamental: en un SELECT con GROUP BY, solo puedes incluir columnas que estén en el GROUP BY o que estén dentro de una función de agregación. No puedes mezclar columnas individuales con grupos.

```
# Consulta 4: resumen de ingresos por ruta
cur.execute('''
    SELECT    id_ruta,
              COUNT(*)                               AS num_vuelos,
              ROUND(AVG(precio_cop), 0)              AS
precio_promedio,
              SUM(precio_cop * asientos_vendidos)    AS
ingresos_totales
    FROM      vuelos
    GROUP BY id_ruta
    ORDER BY ingresos_totales DESC
''')
for fila in cur.fetchall():
    print(fila)
```

```
(5, 2, 502500.0, 106860000) -- Bogotá-Barranquilla
(3, 2, 465000.0, 92400000)  -- Bogotá-Cartagena
(1, 3, 182333.0, 78856000)  -- Bogotá-Medellín
(2, 2, 215000.0, 49910000)  -- Bogotá-Cali
(4, 1, 380000.0, 28500000)  -- Medellín-Cartagena
```

#### Interpretación: Paso 3

**Interpretación técnica:** precio\_cop \* asientos\_vendidos calcula el ingreso de cada vuelo dentro del SUM, que luego los acumula por grupo. El alias AS le da un nombre legible al resultado calculado. ROUND(..., 0) redondea a cero decimales.

**Interpretación de negocio:** la ruta Bogotá-Barranquilla genera los mayores ingresos totales (106.9M COP) a pesar de tener solo 2 vuelos, gracias a su alto precio promedio (502.500 COP). La ruta Bogotá-Medellín tiene el mayor número de vuelos (3) pero ingresos moderados por su precio bajo. Esta tabla es exactamente el tipo de reporte que un director

de ingresos de Avianca consultaría para decidir dónde agregar capacidad.

## Paso 4: HAVING — filtrar grupos

HAVING filtra el resultado del GROUP BY, igual que WHERE filtra filas individuales. La diferencia es crucial: WHERE se aplica antes de agrupar (sobre filas individuales), HAVING se aplica después de agrupar (sobre los grupos ya calculados). Por eso HAVING puede usar funciones de agregación y WHERE no.

```
# Consulta 5: rutas cuyo ingreso total supera 80 millones COP
cur.execute('''
    SELECT    id_ruta,
              COUNT(*)                               AS num_vuelos,
              SUM(precio_cop * asientos_vendidos)     AS ingresos_totales
    FROM      vuelos
    GROUP BY  id_ruta
    HAVING    ingresos_totales > 80000000
    ORDER BY  ingresos_totales DESC
''')
for fila in cur.fetchall():
    print(fila)
```

```
(5, 2, 106860000)  -- Bogotá-Barranquilla: 106.9M
(3, 2, 92400000)   -- Bogotá-Cartagena: 92.4M
```

### Interpretación: Paso 4

**Diferencia WHERE vs HAVING:** si hubieras escrito WHERE ingresos\_totales > 80000000, SQL daría error porque ingresos\_totales no existe todavía al evaluar WHERE (es un alias creado por SUM). HAVING sí puede usarlo porque se evalúa después del GROUP BY.

**Interpretación de negocio:** solo 2 de 5 rutas superan el umbral de 80M COP en este período. Barranquilla y Cartagena son los mercados de mayor rentabilidad. Avianca podría usar este criterio para priorizar inversión en frecuencias adicionales en estas rutas.

## Paso 5: INNER JOIN — combinar tablas

Un JOIN combina filas de dos tablas usando una condición de unión. El INNER JOIN devuelve solo las filas que tienen coincidencia en ambas tablas. Si un vuelo tiene un id\_ruta que no existe en la tabla rutas, ese vuelo no aparece en el resultado.

```
# Consulta 6: vuelos con nombre de ruta (no solo id)
```

```

cur.execute('''
    SELECT v.id_vuelo, r.origen, r.destino,
           v.fecha, v.precio_cop, v.asientos_vendidos
    FROM   vuelos v
    INNER JOIN rutas r ON v.id_ruta = r.id_ruta
    ORDER BY v.id_vuelo
    LIMIT 5
''')
for fila in cur.fetchall():
    print(fila)

```

```

(1, 'Bogotá', 'Medellín', '2024-03-01', 180000, 142)
(2, 'Bogotá', 'Medellín', '2024-03-02', 195000, 156)
(3, 'Bogotá', 'Cali', '2024-03-01', 220000, 98)
(4, 'Bogotá', 'Cali', '2024-03-03', 210000, 135)
(5, 'Bogotá', 'Cartagena', '2024-03-01', 450000, 88)

```

### Interpretación: Paso 5

**Interpretación técnica:** v y r son alias de tabla (abreviaturas). La condición ON v.id\_ruta = r.id\_ruta le dice a SQL cómo unir las tablas: 'combina cada vuelo con la ruta cuyo id coincida'. LIMIT 5 muestra solo las primeras 5 filas.

**Interpretación de negocio:** en lugar de ver id\_ruta=1, ahora vemos 'Bogotá → Medellín'. El JOIN transforma identificadores numéricos en información legible. Esto es lo que hace que los reportes de negocio sean útiles para personas no técnicas.

## Paso 6: JOIN múltiple y consulta analítica

En la práctica, una consulta útil conecta 3 o 4 tablas. La sintaxis es la misma: se encadenan múltiples INNER JOIN. Esta consulta muestra el historial completo de cada reserva: nombre del pasajero, ruta, clase y fecha.

```

# Consulta 7: reservas con toda la información combinada
cur.execute('''
    SELECT p.nombre, r.origen, r.destino, res.clase, v.fecha
    FROM   reservas res
    INNER JOIN vuelos v ON res.id_vuelo = v.id_vuelo
    INNER JOIN rutas r ON v.id_ruta = r.id_ruta
    INNER JOIN pasajeros p ON res.id_pasajero = p.id_pasajero
    ORDER BY p.nombre
''')
for fila in cur.fetchall():
    print(fila)

```



```
(
  'Andrés Torres', 'Bogotá', 'Medellín', 'Ejecutiva', '2024-03-02')
(
  'Andrés Torres', 'Bogotá', 'Barranquilla', 'Ejecutiva', '2024-03-01')
(
  'Camilo Vargas', 'Bogotá', 'Cartagena', 'Ejecutiva', '2024-03-04')
(
  'Carlos Mejía', 'Bogotá', 'Medellín', 'Ejecutiva', '2024-03-01')
(
  'Carlos Mejía', 'Medellín', 'Cartagena', 'Ejecutiva', '2024-03-02')
(
  'Diego Ramírez', 'Bogotá', 'Cali', 'Económica', '2024-03-03')
(
  'Laura Ospina', 'Bogotá', 'Medellín', 'Económica', '2024-03-01')
(
  'Paula Gómez', 'Bogotá', 'Barranquilla', 'Económica', '2024-03-05')
(
  'Sofía Herrera', 'Bogotá', 'Cartagena', 'Económica', '2024-03-01')
(
  'Valentina Cruz', 'Bogotá', 'Cali', 'Económica', '2024-03-01')
```

### Interpretación: Paso 6

**Interpretación técnica:** reservas es la tabla puente (tabla de intersección) entre vuelos y pasajeros. Sin ella, no habría forma de conectar ambas tablas. El encadenamiento de tres JOINS sigue la lógica: reserva → vuelo → ruta y reserva → pasajero.

**Interpretación de negocio:** los pasajeros Gold LifeMiles (Carlos Mejía, Andrés Torres, Camilo Vargas) viajan exclusivamente en clase Ejecutiva. Los pasajeros Basic viajan en Económica. Andrés Torres y Carlos Mejía tienen 2 reservas cada uno. Esta tabla es la base de cualquier análisis de comportamiento de viajero frecuente.

## Paso 7: Consulta analítica con GROUP BY + JOIN + HAVING

Esta es la consulta más completa de la guía: combina JOINS, agrupaciones, métricas calculadas y filtrado por HAVING. Es el tipo de consulta que encontrarás en reportes de negocio reales.

```
# Consulta 8: ocupación por ruta (% de asientos vendidos)
cur.execute('''
    SELECT      r.origen, r.destino,
               COUNT(v.id_vuelo)
    AS vuelos,
               ROUND(100.0 * SUM(v.asientos_vendidos)
                     / SUM(v.asientos_totales), 1)
    AS ocupacion_pct,
               ROUND(AVG(v.precio_cop), 0)
    AS precio_promedio
    FROM        vuelos v
    INNER JOIN  rutas r ON v.id_ruta = r.id_ruta
    GROUP BY   r.id_ruta
    HAVING      ocupacion_pct > 75
    ORDER BY   ocupacion_pct DESC
```

```
'''
for fila in cur.fetchall():
    print(fila)
```

```
('Bogotá', 'Medellín', 3, 84.5, 182333.0)
('Bogotá', 'Cartagena', 2, 82.5, 465000.0)
('Bogotá', 'Barranquilla', 2, 81.9, 502500.0)
('Bogotá', 'Cali', 2, 77.7, 215000.0)
```

### Interpretación: Paso 7

**Interpretación técnica:** el uso de 100.0 (no 100) fuerza la división de punto flotante en SQLite. Con 100 (entero) la división entera daría resultados incorrectos. GROUP BY r.id\_ruta es correcto aunque el SELECT muestre origen y destino, porque estos dependen funcionalmente del id\_ruta.

**Interpretación de negocio:** 4 de 5 rutas tienen más del 75% de ocupación. Bogotá-Medellín lidera con 84.5% de ocupación a pesar de ser la ruta de menor precio (182.333 COP). La ruta Medellín-Cartagena (ocupación < 75%) queda excluida del reporte, señal de menor demanda o sobreoferta de asientos.

## Paso 8: LEFT JOIN — incluir filas sin coincidencia

El LEFT JOIN devuelve TODAS las filas de la tabla izquierda (la que aparece antes del JOIN), más las coincidencias de la tabla derecha. Donde no hay coincidencia, las columnas de la tabla derecha aparecen como NULL. Es útil para detectar registros huérfanos.

```
# Consulta 9: pasajeros que NO tienen ninguna reserva
cur.execute('''
    SELECT  p.nombre, p.categoria, res.id_reserva
    FROM    pasajeros p
    LEFT JOIN reservas res ON p.id_pasajero = res.id_pasajero
    WHERE   res.id_reserva IS NULL
''')
resultado = cur.fetchall()
if resultado:
    for fila in resultado: print(fila)
else:
    print('Todos los pasajeros tienen al menos una reserva.')
```

```
Todos los pasajeros tienen al menos una reserva.
```

### Interpretación: Paso 8

**Interpretación técnica:** el patrón LEFT JOIN ... WHERE tabla\_derecha.id IS NULL es la forma estándar de encontrar registros sin correspondencia. En este caso, todos los pasajeros tienen al menos una reserva, así que el resultado está vacío.

**Interpretación de negocio:** en una base de datos de producción, este tipo de consulta se usa para auditorías: pasajeros LifeMiles sin vuelos registrados, productos en catálogo sin ventas, empleados sin nómina procesada. Es una herramienta de control de calidad de datos.

## SECCIÓN 3 | Cheat Sheet SQL

| Cláusula / Función | Sintaxis                               | Para qué sirve                                                          |
|--------------------|----------------------------------------|-------------------------------------------------------------------------|
| SELECT             | SELECT col1, col2 FROM tabla           | Selecciona columnas. * selecciona todas.                                |
| WHERE              | WHERE condicion                        | Filtra filas antes de agrupar. Usa =, >, <, AND, OR, LIKE, BETWEEN, IN. |
| ORDER BY           | ORDER BY col ASC/DESC                  | Ordena el resultado. ASC = ascendente (defecto), DESC = descendente.    |
| LIMIT              | LIMIT n                                | Devuelve solo las primeras n filas.                                     |
| GROUP BY           | GROUP BY col                           | Agrupar filas con el mismo valor. Usar con COUNT, SUM, AVG, MIN, MAX.   |
| HAVING             | HAVING condicion_agregada              | Filtra grupos después del GROUP BY. Puede usar alias del SELECT.        |
| INNER JOIN         | ... INNER JOIN tabla2 ON t1.id = t2.id | Devuelve solo filas con coincidencia en ambas tablas.                   |
| LEFT JOIN          | ... LEFT JOIN tabla2 ON t1.id = t2.id  | Devuelve todas las filas de la tabla izquierda + coincidencias.         |
| COUNT(*)           | SELECT COUNT(*) FROM tabla             | Cuenta el total de filas del grupo.                                     |
| SUM(col)           | SELECT SUM(precio) FROM ventas         | Suma los valores de la columna en el grupo.                             |
| AVG(col)           | SELECT AVG(precio) FROM ventas         | Promedio de la columna en el grupo.                                     |
| ROUND(x, n)        | ROUND(AVG(precio), 2)                  | Redondea a n decimales.                                                 |
| IS NULL            | WHERE col IS NULL                      | Detecta valores nulos. No usar = NULL.                                  |
| AS                 | SELECT col AS alias                    | Renombra una columna en el resultado.                                   |

## SECCIÓN 4 | Ejercicio Propuesto Autónomo



### Contexto: Tigo Colombia — Análisis de planes y consumo de datos

Tigo Colombia es una operadora de telecomunicaciones con sede en Bogotá, parte del grupo Millicom International Cellular. Opera en telefonía móvil, internet fijo, televisión y servicios empresariales en más de 700 municipios del país. Es una de las principales operadoras en segmentos de mercado masivo y prepago.

Caso de uso: el equipo de datos de Tigo necesita analizar el comportamiento de consumo de sus clientes por región y tipo de plan para optimizar la oferta de paquetes.

Crea la siguiente base de datos en SQLite:

| Tabla    | Columnas clave                                                                             |
|----------|--------------------------------------------------------------------------------------------|
| regiones | id_region (PK), nombre, departamento, tipo (Urbana/Rural)                                  |
| planes   | id_plan (PK), nombre_plan, tipo (Prepago/Pospago), precio_mensual, datos_gb                |
| clientes | id_cliente (PK), id_region (FK), id_plan (FK), fecha_vinculacion, estado (Activo/Inactivo) |
| consumos | id_consumo (PK), id_cliente (FK), mes, datos_consumidos_gb, llamadas_minutos               |

### Puntos del ejercicio

#### Punto 1 — SELECT y WHERE

- Selecciona todos los clientes con estado 'Activo' que se vincularon antes de 2024-01-01.
- Selecciona los planes de tipo Pospago con precio mensual mayor a 60.000 COP.
- En tu celda Markdown: ¿cuántos clientes activos tiene cada región según tus datos? ¿Qué implica esto para el negocio?

#### Punto 2 — GROUP BY

- Calcula el promedio de datos consumidos (GB) por mes, agrupado por tipo de plan.
- Calcula el total de minutos de llamadas por región, ordenado de mayor a menor.
- ¿Qué región consume más datos? ¿Y más minutos de voz?

### Punto 3 — HAVING

- Filtra las regiones donde el consumo promedio de datos supera 8 GB por cliente.
- Filtra los planes donde el total de clientes activos es mayor a 2.
- Explica por qué no podrías usar WHERE en lugar de HAVING en estas consultas.

### Punto 4 — JOINS

- Usa INNER JOIN para mostrar: nombre del cliente, nombre de la región, nombre del plan y consumo del último mes.
- Usa LEFT JOIN para encontrar clientes que NO tienen ningún registro de consumo.
- ¿Qué significaría un cliente sin consumo para el modelo de negocio de Tigo?

### Punto 5 — Consulta analítica integrada

- Escribe una consulta que muestre, por tipo de plan y por región: total de clientes activos, promedio de datos consumidos, y total de ingresos estimados (clientes activos × precio del plan). Filtra solo los grupos con más de 1 cliente activo.
- En tu celda Markdown: ¿qué combinación tipo-de-plan/región es más rentable para Tigo?



#### Criterio de autoevaluación

Tu trabajo está completo si: (1) cada consulta produce el resultado correcto y puedes explicar por qué cada cláusula es necesaria; (2) diferencias entre WHERE y HAVING explicadas con ejemplos concretos de tus datos; (3) la consulta del Punto 5 combina JOIN, GROUP BY, agregaciones y HAVING en una sola consulta funcional.